GENERATIVE AI & LLMS

Vector Databases (FAISS & ChromaDB)

Turn meaning into geometry — embeddings, ANN indexes, and the FAISS vs ChromaDB playbook for production semantic search.

vector-db | FAISS | ChromaDB | embeddings | ANN | HNSW | similarity-search

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. 1. What a Vector Database Is and Why It Matters
2. 2. Embeddings and Similarity
3. 3. ANN Indexes: Flat, IVF, HNSW
4. 4. FAISS: the In-Process Speed Engine
5. 5. ChromaDB: the Batteries-Included Store
6. 6. A Worked Mini-Example: Semantic FAQ Search
7. 7. Cheat-Sheet and Engine Comparison
8. 8. Persistence and Scaling
9. 9. Best Practices
10. 10. Common Pitfalls
11. 11. Interview Questions & Answers
12. 12. Further Learning

A vector database turns *meaning* into geometry. Text, images, and audio become high-dimensional points; "find things similar to this" becomes "find the nearest points." Master embeddings, approximate-nearest-neighbour (ANN) indexes, and two workhorse engines — FAISS for raw in-process speed and ChromaDB for a batteries-included store — and you can build the retrieval layer behind RAG, semantic search, recommendations, and deduplication.

1. What a Vector Database Is and Why It Matters

Classic databases index *exact* values: a B-tree on `email = 'x@y.com'`. But "which support tickets are *about the same problem* as this one?" has no exact key — the match is semantic, not lexical. The trick is to convert each item into an **embedding**: a dense vector (often 384–3072 floats) produced by a neural network such that *semantically similar inputs land close together* in the vector space. Searching becomes a geometry problem: given a query vector, return the `k` stored vectors with the smallest distance.

SEMANTIC RECALL

"car won't start" matches "engine fails to crank" even with zero shared words — meaning, not keywords.

MULTIMODAL

The same geometry works for images, audio, and code once each modality has an embedding model.

SCALES SUB-LINEARLY

ANN indexes answer queries over millions of vectors in milliseconds, far below a brute-force scan.

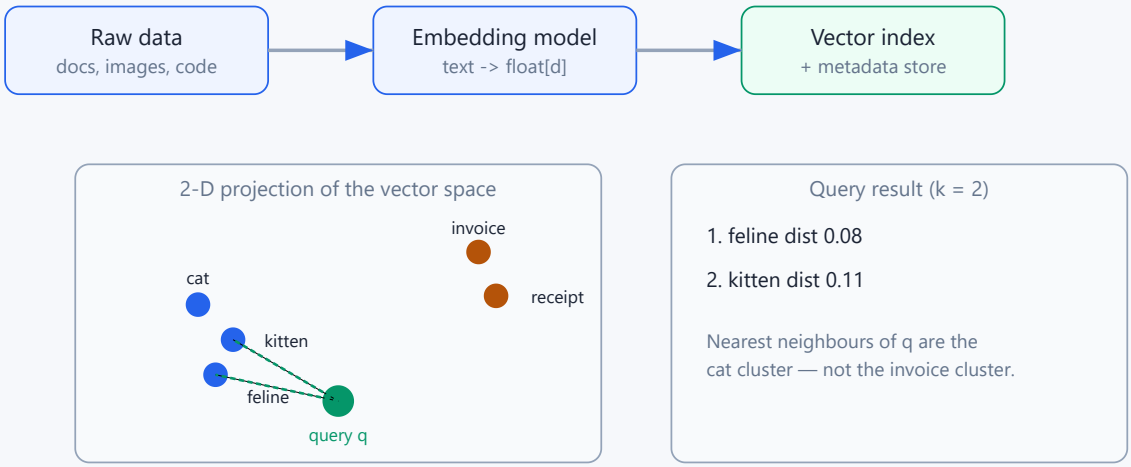


Fig 1 — Ingest pipeline (top) and the geometric view (bottom): similar meanings cluster, and k-NN around the query returns the right neighbours.

Mental model

An embedding model is a *meaning compressor*; the vector DB is a *spatial index* over its output. The DB never understands text — it only does fast geometry. Retrieval quality is therefore capped by your embedding model, not your index.

2. Embeddings and Similarity

An embedding is a point in `R^d`. Three distance/similarity measures dominate; choosing the right one — and matching it to how the model was trained — is the single most common source of silent bugs.

COSINE SIMILARITY

Angle between vectors, ignores magnitude. The default for text embeddings. Range [-1, 1]; higher = more similar.

DOT (INNER) PRODUCT

Angle *and* magnitude. Equals cosine when vectors are L2-normalised. Used by many retrieval models trained with it.

EUCLIDEAN (L2)

Straight-line distance. Lower = more similar. On normalised vectors, L2 and cosine rank identically.

✦ **Normalise and the choice collapses**

If you L2-normalise every vector (unit length), then cosine, dot product, and L2 produce the *same ranking*. Normalising up front lets you use the fastest metric the index supports without changing results.

PYTHON

```python
import numpy as np

def l2_normalize(v: np.ndarray) -> np.ndarray:
    """Scale each row to unit length so cosine == dot product."""
    norm = np.linalg.norm(v, axis=-1, keepdims=True)
    return v / np.clip(norm, 1e-12, None)   # avoid divide-by-zero

a = np.array([1.0, 2.0, 2.0])
b = np.array([2.0, 4.0, 4.0])              # same direction, 2x magnitude

cos = a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
print(round(cos, 4))                        # 1.0  → identical meaning

an, bn = l2_normalize(a), l2_normalize(b)
print(round(float(an @ bn), 4))             # 1.0  → dot == cosine after norm
print(round(float(np.linalg.norm(an - bn)), 4))  # 0.0 → L2 agrees
```

Producing embeddings is a model call. The de-facto open default is `sentence-transformers`; hosted options (OpenAI `text-embedding-3`, Cohere, Voyage) trade local control for quality and zero ops.

PYTHON

```python
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("BAAI/bge-small-en-v1.5")   # 384-dim, fast, strong
docs = ["The engine won't crank", "Payment receipt attached", "My car fails to start"]

emb = model.encode(docs, normalize_embeddings=True)      # normalise at the source
print(emb.shape)                                         # (3, 384)
# doc[0] and doc[2] are semantically close despite no shared words
print(round(float(emb[0] @ emb[2]), 3), round(float(emb[0] @ emb[1]), 3))
```

⚠ **Asymmetric models need prefixes**

Many retrieval models (BGE, E5, GTE) were trained with instruction prefixes — e.g. embed queries as `"query: ..."` and documents as `"passage: ..."`. Skipping the prefix quietly degrades recall. Always read your model card.

3. ANN Indexes: Flat, IVF, HNSW

Exact nearest-neighbour search ("Flat") compares the query against *every* vector — O(N·d) per query. Perfect recall, but linear cost. **Approximate** indexes trade a sliver of recall for orders-of-magnitude speed. The three archetypes below cover almost every real system.

Index	How it searches	Build cost	Query speed	Recall	Memory	Best for
Flat (brute force)	Compare to all N	None	Slow (linear)	100% exact	Low (raw vectors)	< ~100k vectors; a correctness baseline
IVF (inverted file)	Cluster, probe nearest cells	Needs training	Fast	Tunable (nprobe)	Medium	Millions, static-ish data
HNSW (graph)	Greedy walk on a layered graph	Higher	Very fast	High, tunable (ef)	High (graph links)	Low-latency, high-recall, incremental inserts

IVF — partition the space, search a few partitions

IVF runs k-means to carve the space into `nlist` cells (Voronoi regions). Each vector is filed under its nearest centroid. At query time you only scan the `nprobe` cells nearest the query — not all N. Higher `nprobe` = more cells scanned = better recall, slower query.

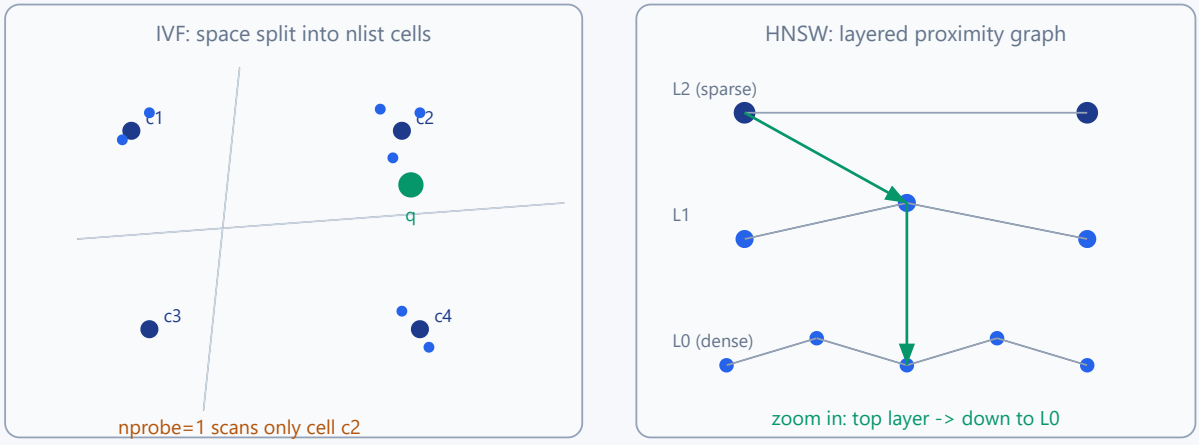


Fig 2 — IVF (left) scans only the nprobe nearest cells; HNSW (right) descends layers, hopping greedily toward the query's neighbourhood.

HNSW — Hierarchical Navigable Small World

HNSW builds a multi-layer graph. The top layer has few nodes with long-range links; lower layers get denser. Search starts at the top, greedily hops to the closest neighbour, then drops a layer and

repeats — like zooming in on a map. Two knobs dominate: `M` (links per node, affects graph quality and memory) at build, and `ef_search` (candidate list size) at query — raise it for recall, lower it for speed.

> ✦ **Default pick**
>
> If unsure, start with **HNSW**: excellent recall/latency, supports incremental inserts (no training step), and is the backbone of most modern vector DBs. Reach for IVF (often `IVF+PQ`) when memory for billions of vectors is the binding constraint.

Compression: Product Quantization (PQ)

Raw `float32` vectors are heavy: 1M × 768-dim ≈ 3 GB. **Product Quantization** splits each vector into m sub-vectors and replaces each with a 1-byte codebook id, shrinking storage ~10–50×. It is lossy — pair it with a reranking pass over exact vectors when precision matters. In FAISS this is the `IVF…,PQ` family (e.g. `IVF4096,PQ64`).

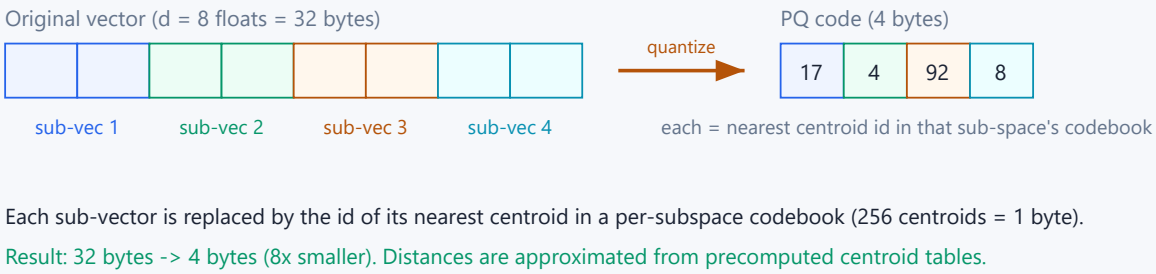


Fig 3 — Product Quantization: split the vector, map each slice to a codebook id. Storage collapses; distances become fast table lookups (lossy).

4. FAISS: the In-Process Speed Engine

FAISS (Facebook AI Similarity Search) is a C++ library with Python bindings. It is *not* a database — no persistence API beyond file read/write, no metadata filtering, no server. It is a blazing-fast index you embed in your own process. You own IDs, metadata, and storage.

BASH

```
pip install faiss-cpu            # CPU build (most common)
# pip install faiss-gpu-cu12     # CUDA build for GPU search, if you have the hardware
```

Flat index — exact baseline

PYTHON

```python
import faiss, numpy as np

d = 384                                  # embedding dimension
rng = np.random.default_rng(0)
xb = rng.random((10_000, d)).astype("float32")   # database vectors
faiss.normalize_L2(xb)                   # in-place; now inner product == cosine

index = faiss.IndexFlatIP(d)             # IP = inner product (cosine on unit vectors
index.add(xb)                            # no training needed for Flat
print(index.ntotal)                      # 10000

xq = rng.random((5, d)).astype("float32")
faiss.normalize_L2(xq)
D, I = index.search(xq, k=3)             # D=similarities, I=neighbour ids
print(I[0])                              # ids of the 3 nearest for query 0
print(D[0])                              # their cosine similarities (desc)
```

IP vs L2

`IndexFlatIP` returns *higher* = *better* (inner product). `IndexFlatL2` returns *lower* = *better* (squared Euclidean). Pick IP + normalised vectors for cosine-style retrieval.

IVF index — trained, scalable

PYTHON

```python
nlist = 100                                  # number of Voronoi cells
quantizer = faiss.IndexFlatIP(d)             # assigns vectors to cells
index = faiss.IndexIVFFlat(quantizer, d, nlist, faiss.METRIC_INNER_PRODUCT)

assert not index.is_trained
index.train(xb)                              # k-means over a representative sample
index.add(xb)

index.nprobe = 10                            # scan 10 of 100 cells (recall vs speed)
D, I = index.search(xq, k=3)
# Rule of thumb: train on ≥ ~30–256 * nlist vectors for stable centroids.
```

HNSW index — graph, no training

PYTHON

```python
index = faiss.IndexHNSWFlat(d, 32)           # M = 32 links per node
index.hnsw.efConstruction = 200              # build-time quality
index.add(xb)                                # incremental; no train() required
index.hnsw.efSearch = 64                     # query-time recall knob
D, I = index.search(xq, k=3)
```

Keeping IDs and persisting

FAISS stores only vectors by position. To attach your own ids, wrap with `IndexIDMap`; keep metadata in a side dict/DB keyed by that id. Persist with `write_index` / `read_index`.

PYTHON

```python
index = faiss.IndexIDMap(faiss.IndexFlatIP(d))
ids = np.array([1001, 1002, 1003], dtype="int64")
index.add_with_ids(xb[:3], ids)              # now search returns YOUR ids

faiss.write_index(index, "my.index")         # persist to disk
index2 = faiss.read_index("my.index")        # reload later

# Metadata lives beside the index — you manage it:
meta = {1001: {"title": "Brakes guide", "lang": "en"}, 1002: {…}, 1003: {…}}
```

⚠ **No native filtering**

FAISS cannot do "nearest vectors *where lang=en*" in one call. Options: build separate indexes per filter value, over-fetch then filter in Python (cheap, approximate), or use an `IDSelector` (newer FAISS) to restrict the search set. If filtering is central, prefer ChromaDB or a server DB.

5. ChromaDB: the Batteries-Included Store

ChromaDB is an open-source vector *database*: it bundles the index (HNSW under the hood), a metadata store, optional on-disk persistence, and can even call an embedding function for you. You add raw documents; Chroma embeds, indexes, and lets you filter by metadata — far less glue code than FAISS.

BASH

```
pip install chromadb
```

PYTHON

```python
import chromadb

# Persistent client: data survives restarts under ./chroma_store
client = chromadb.PersistentClient(path="./chroma_store")
# client = chromadb.Client()          # ephemeral, in-memory (tests)

collection = client.get_or_create_collection(
    name="support_docs",
    metadata={"hnsw:space": "cosine"}, # distance: cosine | l2 | ip
)

# Chroma embeds documents for you with a default model (or supply your own)
collection.add(
    ids=["d1", "d2", "d3"],
    documents=["The engine won't crank",
               "Payment receipt attached",
               "My car fails to start"],
    metadatas=[{"topic": "engine", "lang": "en"},
               {"topic": "billing", "lang": "en"},
               {"topic": "engine", "lang": "en"}],
)

res = collection.query(
    query_texts=["car won't turn over"],
    n_results=2,
    where={"topic": "engine"},         # metadata filter, applied natively
)
print(res["ids"], res["distances"])    # d3 / d1 — the engine docs
```

Bring your own embeddings

Decouple embedding from storage when you want a specific model or precomputed vectors — pass `embeddings=` on add and `query_embeddings=` on query. Keep the *same* model for both or results are garbage.

PYTHON

```python
from chromadb.utils import embedding_functions

ef = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="BAAI/bge-small-en-v1.5")
col = client.get_or_create_collection("docs", embedding_function=ef)

# Or supply vectors directly (no embedding_function needed):
col.add(ids=["x"], embeddings=[[0.1, 0.2, 0.3]], metadatas=[{"k": 1}])
col.query(query_embeddings=[[0.1, 0.2, 0.25]], n_results=5)
```

Metadata filtering — `where` and `where_document`

Chroma filters structured metadata with Mongo-style operators and does full-text contains on the document body. Filtering happens *during* the vector search, so you get the k nearest *that also match*.

PYTHON

```python
collection.query(
    query_texts=["refund status"],
    n_results=5,
    where={                                  # boolean metadata predicate
        "$and": [
            {"lang": {"$eq": "en"}},
            {"priority": {"$gte": 3}},
        ]
    },
    where_document={"$contains": "invoice"}, # must contain this substring
)
# Operators: $eq $ne $gt $gte $lt $lte $in $nin $and $or
```

✦ **CRUD you'll actually use**

`collection.get(ids=[ … ])` fetches by id; `collection.update( … )` / `upsert( … )` change vectors or metadata; `collection.delete(ids=[ … ] | where= … )` removes. `upsert` is idempotent — re-ingest safely without duplicates.

6. A Worked Mini-Example: Semantic FAQ Search

End to end: embed a small FAQ, index it, answer a free-text question. Shown in both engines so the contrast is concrete.

PYTHON

```python
# ——— Shared: the knowledge base ———
faq = [
    ("How do I reset my password?",        "account"),
    ("Where can I download my invoice?",   "billing"),
    ("My card was charged twice",          "billing"),
    ("The app crashes on startup",         "bug"),
]
texts = [q for q, _ in faq]
tags  = [t for _, t in faq]

from sentence_transformers import SentenceTransformer
m = SentenceTransformer("BAAI/bge-small-en-v1.5")
vecs = m.encode(texts, normalize_embeddings=True).astype("float32")
q = m.encode(["I need a copy of my bill"], normalize_embeddings=True).astype("float32"
```

PYTHON

```python
# ——— Option A: FAISS (fast, you manage metadata) ———
import faiss, numpy as np
index = faiss.IndexFlatIP(vecs.shape[1])
index.add(vecs)
D, I = index.search(q, k=2)
for rank, (idx, score) in enumerate(zip(I[0], D[0]), 1):
    print(rank, texts[idx], f"({tags[idx]}, sim={score:.3f})")
# 1 Where can I download my invoice? (billing, sim=0.72)
```

PYTHON

```python
# ——— Option B: ChromaDB (filtering + persistence built in) ———
import chromadb
col = chromadb.Client().create_collection("faq", metadata={"hnsw:space": "cosine"})
col.add(ids=[str(i) for i in range(len(texts))],
        embeddings=vecs.tolist(),
        documents=texts,
        metadatas=[{"tag": t} for t in tags])

res = col.query(query_embeddings=q.tolist(), n_results=2,
                where={"tag": "billing"})     # restrict to billing FAQs
print(res["documents"][0])                     # ['Where can I download my invoice?',
```

7. Cheat-Sheet and Engine Comparison

FAISS quick reference

Task	Call
Exact cosine index	`faiss.IndexFlatIP(d)` + `faiss.normalize_L2(x)`
Exact Euclidean index	`faiss.IndexFlatL2(d)`
IVF (scalable)	`faiss.IndexIVFFlat(quant, d, nlist, metric)` + `train()`
HNSW (graph)	`faiss.IndexHNSWFlat(d, M)`, set `efSearch`
Compressed (billions)	`faiss.index_factory(d, "IVF4096,PQ64")`
Custom ids	`faiss.IndexIDMap(base)` + `add_with_ids()`
Save / load	`faiss.write_index(ix, p)` / `faiss.read_index(p)`
Search	`D, I = index.search(xq, k)`

ChromaDB quick reference

Task	Call
Persistent client	`chromadb.PersistentClient(path="...")`
Create / fetch collection	`client.get_or_create_collection(name, metadata={"hnsw:space":"cosine"})`
Insert	`col.add(ids, documents|embeddings, metadatas)`
Idempotent insert	`col.upsert(...)`
Query	`col.query(query_texts|query_embeddings, n_results, where, where_document)`
Fetch by id	`col.get(ids=[...], include=["metadatas","documents"])`
Delete	`col.delete(ids=[...] | where={...})`
Count	`col.count()`

When to use which

Dimension	FAISS	ChromaDB	Server DB (Qdrant / Weaviate / Milvus / pgvector)
Type	Library (in-process)	Embedded DB (+ server mode)	Standalone service
Metadata filtering	Manual / limited	Native, rich	Native, rich
Persistence	Manual file I/O	Built-in	Built-in, durable
Embeds for you	No	Yes (optional)	Sometimes
Max scale	Billions (with PQ/GPU)	Millions comfortably	Billions, sharded/replicated
Concurrency / multi-node	You build it	Single-node focus	Clustering, HA
Sweet spot	Max control + raw speed, research, embedding in an app	Prototypes to mid prod, RAG with filters	Large-scale production with SLAs

8. Persistence and Scaling



PERSISTENCE

FAISS: `write_index` / `read_index` — one file, you handle metadata separately and keep them in sync. Chroma: `PersistentClient(path=...)` flushes a SQLite + index store automatically. Always snapshot vectors *and* metadata together.

MEMORY BUDGET

float32 bytes ≈ N × d × 4. 10M × 768 ≈ 30 GB raw. HNSW adds graph links (~M × 8 bytes/node). If it won't fit RAM, switch to IVF+PQ or a disk-backed/sharded server DB.

SHARDING

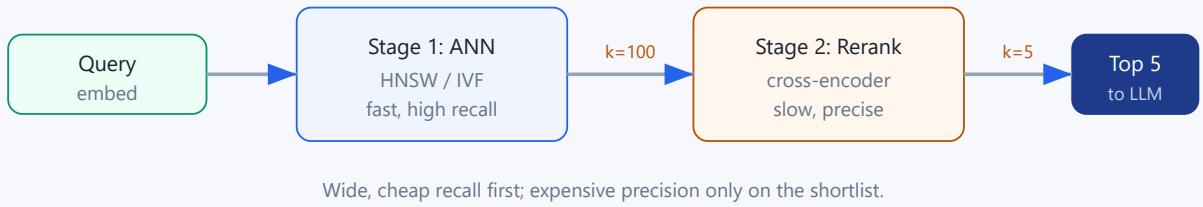
Split vectors across shards, query all, merge top-k. Horizontal scale for both throughput and dataset size. Server DBs (Milvus, Qdrant) automate this; with FAISS you orchestrate it.

FRESHNESS

HNSW supports live inserts; IVF benefits from periodic retraining as the distribution drifts. Rebuild/compact on a schedule, and version your index so you can roll back.

✦ **Two-stage retrieval**

For top quality, retrieve a wide candidate set (e.g. k=100) from the ANN index, then **rerank** with a slower cross-encoder for the final k=5. The ANN layer gives recall; the reranker gives precision. This also hides small ANN recall losses from compression.



Wide, cheap recall first; expensive precision only on the shortlist.

Fig 4 — Two-stage retrieval: a fast ANN index proposes many candidates; a precise reranker trims them to the final answer set.

9. Best Practices

✓ **Do**

Normalise vectors and set the index metric to match how the embedding model was trained. Use the *same* model (and prefix convention) for indexing and querying. Store a stable external id and keep metadata beside vectors.

✓ **Do**

Measure recall against a Flat baseline on a held-out query set before trusting an ANN config. Tune `nprobe` / `efSearch` to a target recall, then freeze it.

✦ **Chunk deliberately**

For documents, embed coherent chunks (sentences/paragraphs with slight overlap), not whole files. Chunk size is a bigger lever on retrieval quality than index choice.

10. Common Pitfalls

✕ **Metric mismatch**

Indexing with L2 while the model expects cosine (or forgetting to normalise) silently tanks relevance — no error, just bad results. Verify with a known query/answer pair.

⚠ **Dimension drift**

Mixing embeddings from two different models (or model versions) in one index is meaningless geometry. Re-embed everything when you change models.

⚠ **Forgetting to train IVF**

Calling `add()` before `train()` on an IVF index throws; training on too few / unrepresentative vectors gives lopsided cells and poor recall.

⚠ **Treating FAISS as a database**

No durability guarantees, no concurrent writers, no filtering, no auth. If you need those, use a real vector DB. FAISS is an index, not a service.

✕ **Curse of dimensionality illusions**

In high dimensions distances concentrate; a "nearest" neighbour at cosine 0.3 may be noise. Always inspect score distributions and set a relevance threshold.

11. Interview Questions & Answers

> **What is an embedding, and why does it enable semantic search?**
>
> A dense vector produced by a model so that semantically similar inputs map to nearby points in R^d. Because meaning becomes geometry, "find similar items" reduces to nearest-neighbour

search — matching on concept rather than exact tokens.

Cosine vs dot product vs Euclidean — when does the choice matter?

On L2-normalised vectors all three rank results identically, so the choice is purely performance/convenience. It matters when vectors are not normalised: dot product then rewards magnitude, cosine ignores it, and L2 mixes both. Match the metric to how the model was trained.

Explain Flat vs IVF vs HNSW and their trade-offs.

Flat is exact brute force: perfect recall, linear cost — good below ~100k. IVF clusters the space and scans only the nprobe nearest cells: fast and memory-moderate but needs training and tuning, best for large static-ish sets. HNSW is a layered proximity graph giving high recall at very low latency with incremental inserts, at higher memory cost — the usual default.

How does HNSW search actually work?

It keeps a hierarchy of graphs: sparse long-range links on top, dense links at the bottom. Search starts at an entry node in the top layer, greedily moves to the neighbour closest to the query, then descends a layer and repeats, expanding a candidate list of size ef_search at the base. More ef = more recall, more work.

What do nprobe and efSearch control, and how do you tune them?

Both trade recall for latency at query time. nprobe = how many IVF cells to scan; efSearch = HNSW candidate-list width. Tune by sweeping values against a Flat baseline on a held-out query set, pick the smallest value hitting your recall target, then freeze it.

What is Product Quantization and when would you use it?

PQ splits each vector into m sub-vectors and encodes each with a 1-byte codebook id, cutting memory ~10-50x. It's lossy, so you use it when billions of vectors won't fit in RAM, usually as IVF+PQ, and add an exact-vector reranking pass to recover precision.

FAISS vs ChromaDB — when do you choose each?

FAISS is a library: maximum speed and control, but you build persistence, ids, metadata, and filtering yourself. ChromaDB is an embedded database: HNSW index plus metadata filtering, persistence, and optional embedding built in. Choose FAISS for raw performance or embedding inside an app; Chroma for prototypes-to-mid-prod RAG where filtering and persistence save real work.

How do you filter by metadata ("nearest where lang=en")?

In Chroma/Qdrant/Weaviate it's native via a where predicate applied during search. FAISS has no native filter: use per-filter indexes, an IDSelector to restrict the search set, or over-fetch and filter in Python. If filtering is central to your query pattern, prefer a real vector DB.

> **How would you measure and guarantee retrieval quality?**
>
> Build a labelled query/relevant-doc set and compute recall@k / MRR / nDCG. Benchmark the ANN config against a Flat (exact) baseline, tune knobs to a recall target, and monitor score distributions in production. Add a cross-encoder reranker when precision at the top matters.

> **How do you scale a vector store to hundreds of millions of vectors?**
>
> Reduce per-vector cost (IVF+PQ, lower dimension, or quantisation), shard across nodes and merge top-k, replicate for throughput/HA, and move to a server DB (Milvus/Qdrant/Weaviate) that automates sharding, persistence, and failover. Keep an eye on the RAM budget: N x d x 4 bytes for float32.

> **Why might two near-identical queries return different neighbours after re-indexing?**
>
> ANN indexes are approximate and often non-deterministic across rebuilds — centroid initialisation (IVF) or insertion order (HNSW) changes the graph/cells. Recall is statistical, not exact. If you need reproducibility, fix seeds, freeze the index, and validate against a Flat baseline.

> **What is a hybrid search and why combine dense with sparse (BM25)?**
>
> Dense vectors capture semantics but can miss exact terms (names, SKUs, error codes); sparse keyword search nails exact matches but misses paraphrase. Hybrid search runs both and fuses scores (e.g. reciprocal rank fusion), getting the recall of semantics plus the precision of lexical matching.

12. Further Learning

- **Benchmark it yourself**: build Flat, IVF, and HNSW over the same 1M vectors and plot recall@10 vs queries/second — the trade-offs become intuition.
- **Read the papers**: the HNSW paper (Malkov & Yashunin) and the original IVF/PQ work (Jégou et al.) — short and clarifying.
- **FAISS wiki**: the `index_factory` guide and "Guidelines to choose an index" page.
- **Explore server DBs**: Qdrant, Weaviate, Milvus, and `pgvector` — each adds filtering, persistence, and clustering on top of the same ANN ideas.
- **Add a reranker**: wire a cross-encoder (e.g. `bge-reranker`) as a second stage and measure the precision lift.
- **Go multimodal**: index CLIP image embeddings and query with text — the exact same geometry, new modality.